

Contents

Architecture's source of truth	1
Deployment	1
Language consideration for the back-end	1
Scope of the app (Current)	1
Interaction with it (platforms, UIs, etc.)	1
Data storage	2
Language consideration for the front-end: Web UI	2
Language consideration for the front-end: Native Android app	2
Feedback de ChatGPT (procesar):	2
1. Define the data model independently of the API	2
2. Decide whether you actually want distributed data	3
3. Offline-first vs online-first	3
4. API design	4
5. Authentication and authorization	4
6. Synchronization semantics	5
7. Event sourcing vs event-driven architecture	5
8. Background jobs	6
9. Message queue: don't introduce one prematurely	6
10. Database	7
11. Backend language	7
Go	7
Rust	7
Java	7
12. Web frontend	8
13. Android	8
14. Observability	8
15. Configuration and secrets	9
16. Backups and disaster recovery	9
17. Versioning/migrations	9
The architecture I'd personally start with	10
One more thing I'd add: write the architecture down as decisions	11

Architecture's source of truth

Deployment

- Ideally Docker, single container for the back-end.

Language consideration for the back-end

- Initial thought: Java. Don't know if Spring Boot is the right tool for the job, seems too clunky.

Scope of the app (Current)

- To-Do list app, but overly engineered.
- Single-backend, self-hosted server.
- Could consider back-end node-like structure for redundancy.

Interaction with it (platforms, UIs, etc.)

- Web UI for non-mobile users.
- Native Android app for the mobile users.

Data storage

- Having doubts between centralizing it on the server vs. spreading it as JSON files stored on each node and replicated / updated on demand through message queues.

Language consideration for the front-end: Web UI

- Taking into account that the web UI should do some processing and sending / receiving on the client side, most likely JS / TS or PHP.
 - Would desire not using a framework, but it kinda makes life easier to not reinvent the wheel.

Language consideration for the front-end: Native Android app

- Flutter is my main candidate.

Feedback de ChatGPT (procesar):

Yes. The biggest thing I'd add is **to design the system boundaries before choosing the languages/frameworks**. Your current architecture is already pointing toward a fairly interesting system rather than a conventional CRUD to-do app.

I'd explicitly think through these areas:

1. Define the data model independently of the API

Before deciding Java vs Rust vs Go, decide what a task actually *is*.

For example:

- Task
 - ID
 - title
 - description
 - status
 - priority
 - tags
 - creation/update timestamps
 - due date
 - recurrence
 - parent task
 - dependencies
 - attachments?
 - ordering
- List/project
- User
- Device
- Event/change
- Notification
- etc.

Especially important: **IDs and timestamps**.

If you ever want replication/offline operation, you don't want to discover later that your database assumes "the server is the authority for everything."

2. Decide whether you actually want distributed data

This is probably your biggest architectural decision.

I'd strongly recommend:

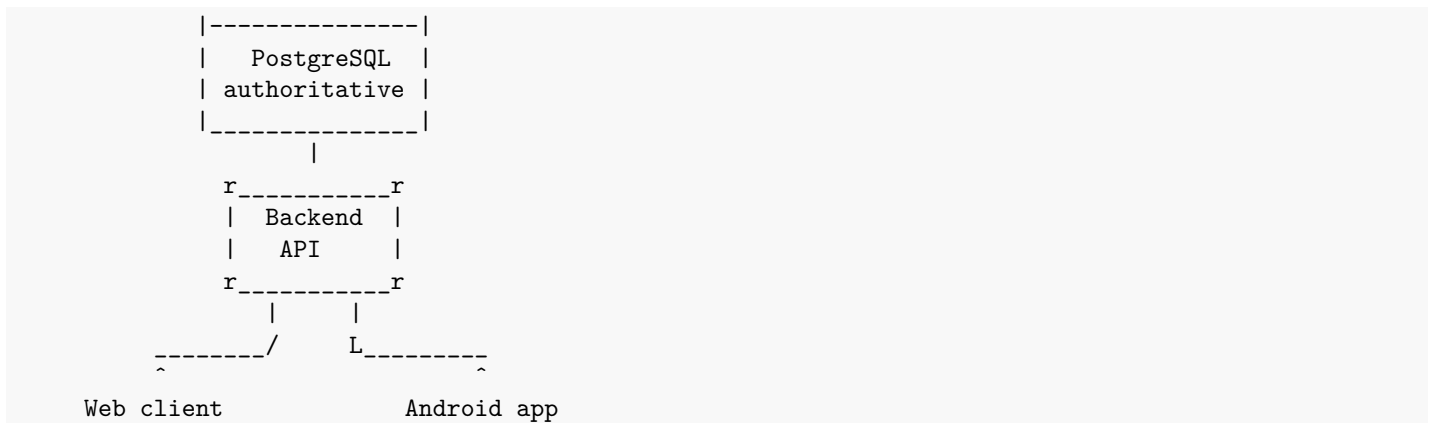
Central authoritative database first. Replication later, if there's a concrete reason for it.

JSON files + message queues + replication sounds cool—and it *is* cool—but it introduces an enormous amount of complexity:

- conflict resolution
- concurrent edits
- ordering
- duplicate messages
- lost messages
- eventual consistency
- node failure
- clock differences
- schema migrations
- deleted objects
- partial replication
- authentication between nodes
- recovery after a node has been offline

You can still architect the application so that adding replication later is possible.

A particularly nice approach would be:



Then internally the backend can have an **event/change-log abstraction**.

If you later decide:

Server A ↔ Server B ↔ Server C

you already have a conceptual place for that functionality.

3. Offline-first vs online-first

This is especially important because you have an Android client.

Ask yourself:

Should the app remain fully usable when the phone has no connection to my server?

If yes, your architecture changes substantially.

You'd probably want:

```
Android
|
|-- Local database
```

```

|
>-- UI
|
|-- Sync engine
    |
    ^
Backend

```

rather than:

```
Android - HTTP API - Backend - DB
```

For example, the user could create five tasks on an airplane and the app synchronizes them when connectivity returns.

That also makes your “JSON replicated between nodes” idea much more relevant—but I’d use a proper local database rather than literally storing your application state as JSON.

4. API design

I’d decide this fairly early.

You have at least:

```

Web -----L
      |
Android ---+--> API
      |
Future CLI /

```

So the API becomes the actual contract of the application.

I’d lean toward **REST + JSON initially**.

For example:

```

GET    /api/v1/tasks
POST   /api/v1/tasks
GET    /api/v1/tasks/{id}
PATCH /api/v1/tasks/{id}
DELETE /api/v1/tasks/{id}

```

But also consider operations rather than just CRUD:

```

POST /api/v1/tasks/{id}/complete
POST /api/v1/tasks/{id}/move
POST /api/v1/tasks/{id}/archive

```

This becomes particularly useful if you eventually introduce an event system.

5. Authentication and authorization

Don’t leave this until the end.

Even if initially there’s only one user.

Think about:

- Authentication
- Sessions/tokens
- Refresh tokens
- Password hashing
- Device registration
- Permissions

- API keys
- Revocation
- HTTPS
- CSRF
- Rate limiting

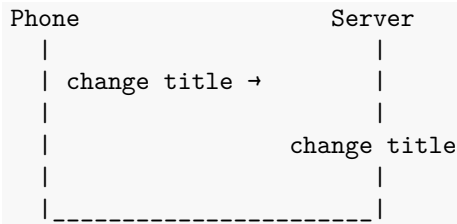
And particularly:

Who is allowed to do what?

If you eventually have multiple people using the system, retrofitting authorization into a CRUD backend can be painful.

6. Synchronization semantics

If you eventually want multiple clients, decide what happens here:



What if both modify the same task?

Possible strategies:

- last-write-wins
- server-wins
- client-wins
- version numbers
- optimistic concurrency
- explicit conflict resolution
- CRDTs

You absolutely don't need CRDTs for v1.

But **versioning entities** is cheap and useful:

```
task.version = 17
```

Client says:

```
PATCH /tasks/123
```

```
If-Version: 17
```

Server rejects it if the task is already version 18.

That gives you a foundation for proper synchronization later.

7. Event sourcing vs event-driven architecture

These are different things.

You might eventually want:

```
TaskCreated
TaskUpdated
TaskCompleted
TaskDeleted
```

That does **not** mean you need full event sourcing.

I'd probably start with:

```
Database
+
Change/Event log
```

rather than making the event log the sole source of truth.

That gives you useful capabilities such as:

```
"What changed since timestamp X?"
```

which becomes extremely useful for synchronization.

8. Background jobs

Your “to-do list on drugs” will probably eventually want things that shouldn’t happen during an HTTP request:

- recurring task generation
- reminders
- email
- push notifications
- cleanup
- synchronization
- imports
- exports

So I'd leave room for:

```
Backend
|__ API
|__ domain logic
|__ database
|__ worker
```

You don't necessarily need a separate container initially.

A single process can have an API server and worker.

9. Message queue: don't introduce one prematurely

You mentioned message queues.

I'd put this on the “**possible future architecture**” list.

Don't start with:

```
API → RabbitMQ → Worker → DB
```

unless you actually have asynchronous workloads that require it.

A database-backed job/event table can take you surprisingly far.

Later:

```
Backend
|
>
Message broker
|__ notifications
|__ replication
|__ integrations
|__ workers
```

becomes a relatively natural evolution.

10. Database

I'd strongly consider **PostgreSQL**.

Not because it's trendy, but because your application sounds like it could eventually have:

- relationships
- transactions
- indexes
- full-text search
- JSON fields where useful
- constraints
- timestamps
- concurrency control
- migrations

I'd avoid designing around JSON files as the primary persistence mechanism.

You can always export the database to JSON.

Going the other direction is much more painful.

11. Backend language

Your instinct about Spring Boot is reasonable.

For this project I'd investigate:

Go

Probably my first alternative to Java.

```
Go
|-- HTTP server
|-- PostgreSQL
|-- migrations
|-- authentication
|-- Docker
```

Very little framework necessary.

Rust

Extremely attractive if the project is also intended as a learning project.

You could use something like:

```
Axum
SQLx
PostgreSQL
Tokio
```

You get a fairly lightweight stack without the Spring ecosystem.

And given that you're already learning Rust, this would make the project substantially more interesting.

Java

Still completely viable.

You don't *have* to use Spring Boot. You could build something considerably lighter with Java's modern HTTP/server ecosystem.

But if your reaction to Spring is already:

“this feels like too much machinery”

I’d listen to that instinct for this particular project.

12. Web frontend

I’d actually avoid PHP here.

Your architecture is naturally:

```
Browser
  ↓
JavaScript/TypeScript
  ↓
REST API
```

You could start with:

```
HTML
CSS
TypeScript
```

and no framework.

Then, if the UI becomes complicated, introduce something like React/Svelte/etc.

That’s a very good place to resist premature abstraction.

And importantly, **TypeScript isn’t the same architectural commitment as choosing a frontend framework.**

You can absolutely write frameworkless TypeScript.

13. Android

Flutter makes sense given your existing experience with it.

I’d make the Android application a proper client rather than letting it know anything about the backend implementation:

```
Flutter
  |
  ~
API client
  |
  ~
Domain models
  |
  ~
Local persistence
```

That will make the eventual offline/synchronization work much easier.

14. Observability

This is one people tend to forget until their homelab application breaks at 02:00.

Have:

- structured logs
- request IDs
- error logging
- health endpoint
- metrics

- database migration version
- application version

Something as simple as:

```
GET /health
GET /health/ready
```

can be extremely useful.

Eventually:

```
Prometheus
Grafana
Loki
```

if you want to go full homelab goblin.

15. Configuration and secrets

Separate:

```
application configuration
```

from:

```
secrets
```

For example:

```
DATABASE_URL
JWT_SECRET
SMTP_PASSWORD
```

should never live in Git.

Docker Compose `.env` is fine for development; you can graduate to Docker secrets or another secret-management solution later.

16. Backups and disaster recovery

Because this is self-hosted, I'd put this surprisingly high on the list.

Ask:

If the Raspberry Pi dies tonight, how much work do I lose?

At minimum:

```
PostgreSQL
|
automated backup
|
another physical/storage location
```

And periodically test **restoring** the backup.

A backup you have never restored is merely a theory.

17. Versioning/migrations

Treat both API and database schemas as things that evolve.

For example:

```
/api/v1/...
```

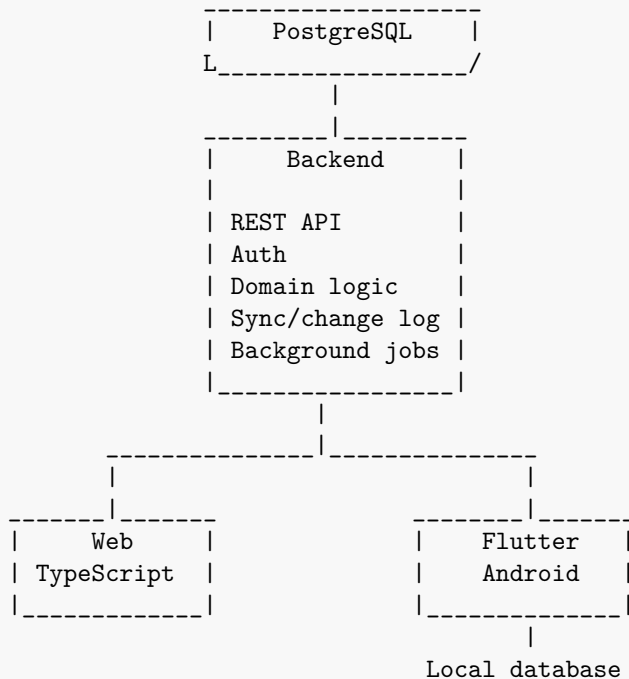
and database migrations:

```
001_initial.sql
002_add_tags.sql
003_add_recurrence.sql
...
```

Don't manually alter production databases.

The architecture I'd personally start with

Given your goals, I'd make v1 roughly:

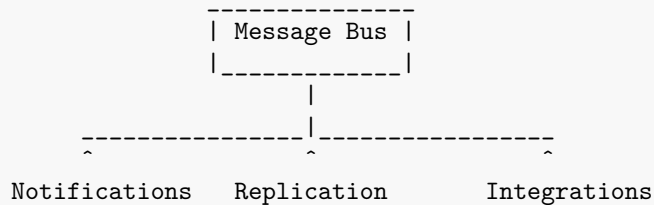


Docker:

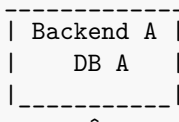
```
docker compose
|-- backend
|-- postgres
```

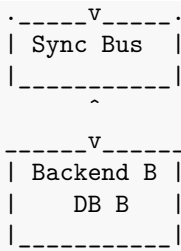
That's it.

Then later:



And *much* later, if you genuinely need it:





The important part is that **v1 doesn't have to pay the complexity cost of v3.**

One more thing I'd add: write the architecture down as decisions

Since you're explicitly trying to design this properly, I'd create something like:

```
docs/
|-- architecture.md
|-- decisions/
|   |-- 001-backend-language.md
|   |-- 002-database.md
|   |-- 003-api-style.md
|   |-- 004-sync-strategy.md
|-- api/
    |-- openapi.yaml
```

Each decision can be tiny:

```
# ADR-002: PostgreSQL as primary persistence

## Context
The application requires relational data...

## Decision
PostgreSQL is the authoritative persistence layer.

## Consequences
- Transactions are available.
- JSON exports remain possible.
- Multi-node replication is not part of v1.
```

That way, six months from now when you're staring at the project thinking *"why the fuck did I choose this?"*, you have an actual reason rather than archaeology through Git commits.

If I were making the next design step, I would not choose Java/Go/Rust yet. I'd first nail down **the domain model + synchronization/offline requirements + API contract**. Those three decisions will make the appropriate backend architecture much clearer.